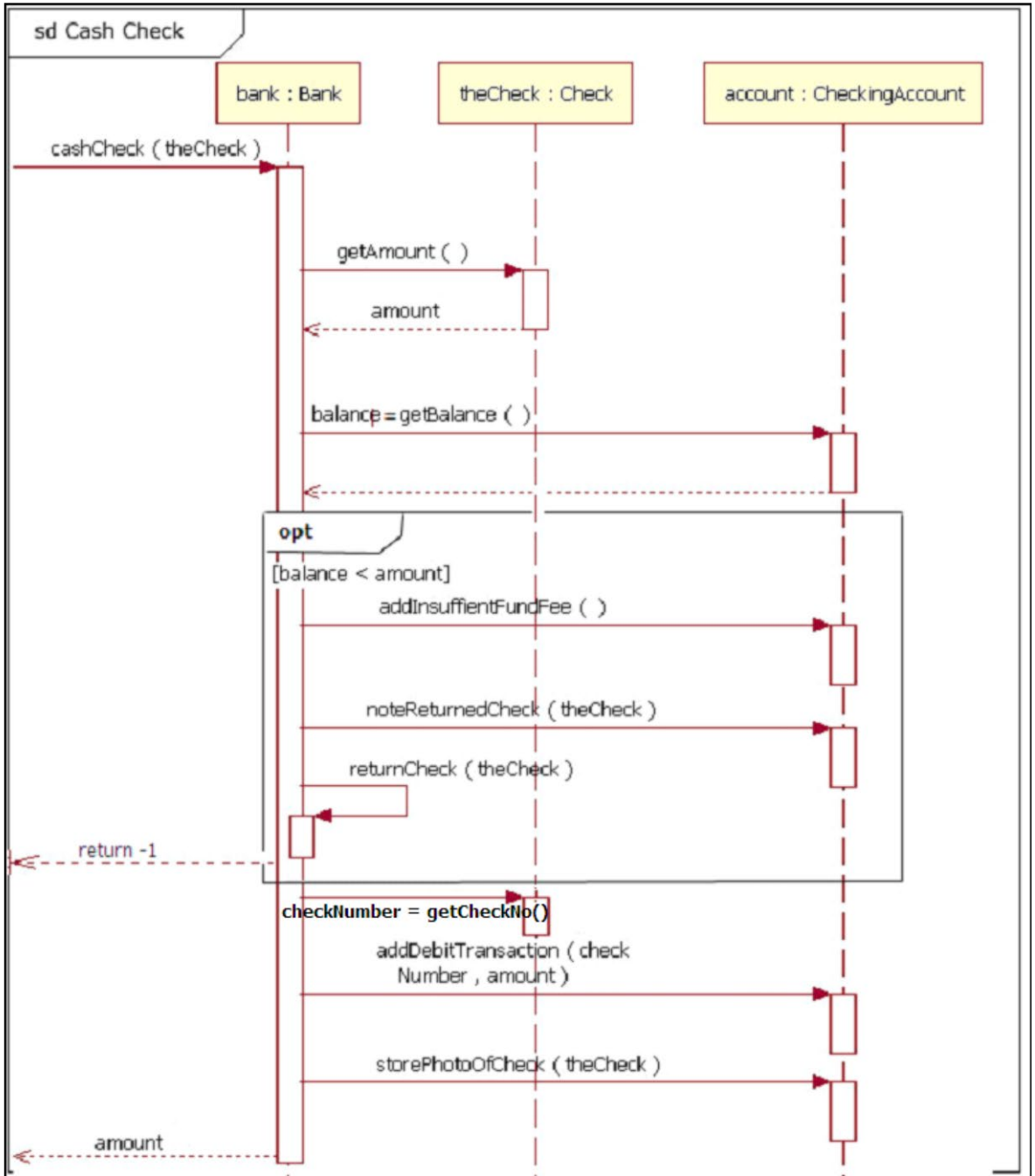
Tutorial 7

- Understand UML **sequence diagram**
  - Behavioural diagram
  - Shows the interaction between objects; how objects communicates between each other and how they pass messages

Q1

- The **UML Sequence Diagram** in Figure Q shows the objects interaction of the “Cash Check” scenario flow. Using the details depicted in the diagram, write the preliminary codes of the JAVA classes for all the participating objects shown. You may make appropriate assumptions on the method parameters and return types.


sd Cash Check
bank : Bank
theCheck : Check
account : CheckingAccount
cashCheck (theCheck)
getAmount ( )
amount
balance = getBalance ( )
opt
[balance < amount]
addInsuffientFundFee ( )
noteReturnedCheck (theCheck)
returnCheck (theCheck)
return -1
checkNumber = getCheckNo()
addDebitTransaction (check Number , amount)
storePhotoOfCheck (theCheck)
amount

These diagrams attempt to show, for some specific use-case or some common interaction:

- what objects are involved
- what operations are involved, and on which objects
- what the sequence of operations is

sequence diagram

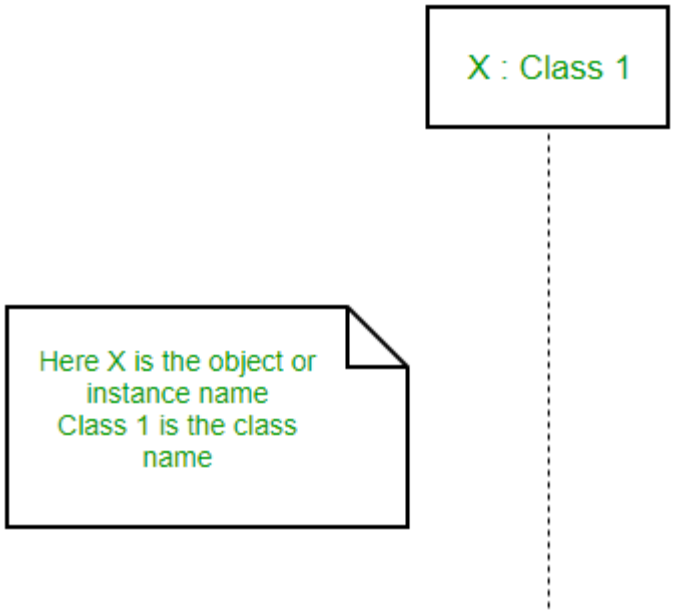
- Sequence diagrams, commonly used by developers, model the interactions between objects in a single use case. They illustrate how the different parts of a system interact with each other to carry out a function, and the order in which the interactions occur when a particular use case is executed.

- In simpler words, a sequence diagram shows different parts of a system work in a ‘sequence’ to get something done.

https://creately.com/blog/diagrams/sequence-diagram-tutorial/#Draw

Lifelines

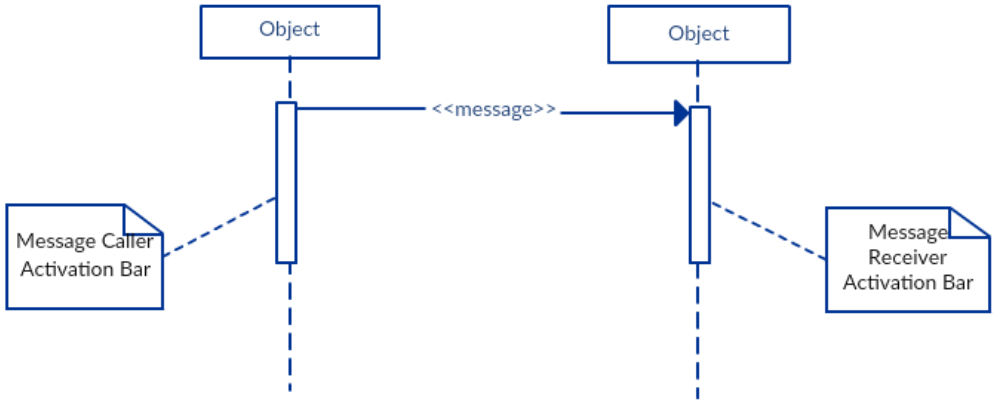
- A sequence diagram is made up of several of these lifeline notations that should be arranged horizontally across the top of the diagram. No two lifeline notations should overlap each other. They represent the different objects or parts that interact with each other in the system during the sequence.
- The standard in UML for naming a lifeline follows the following format – Instance Name : Class Name



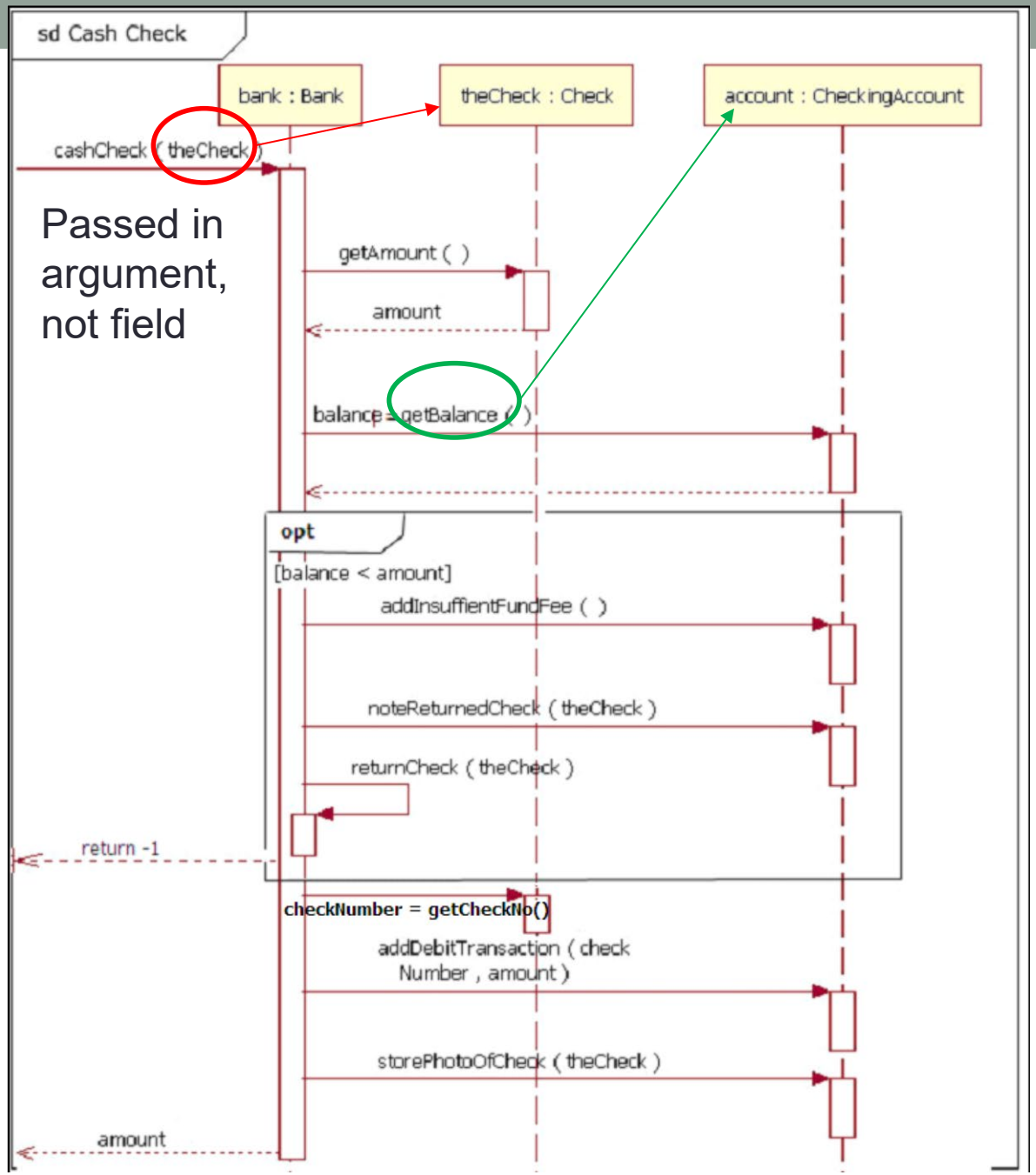
Activation Bars

- Activation bar is the box placed on the lifeline. It is used to indicate that an object is active (or instantiated) during an interaction between two objects. The length of the rectangle indicates the duration of the objects staying active.

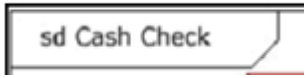
- In a sequence diagram, an interaction between two objects occurs when one object sends a message to another. The use of the activation bar on the lifelines of the Message Caller (the object that sends the message) and the Message Receiver (the object that receives the message) indicates that both are active/is instantiated during the exchange of the message.



Q2



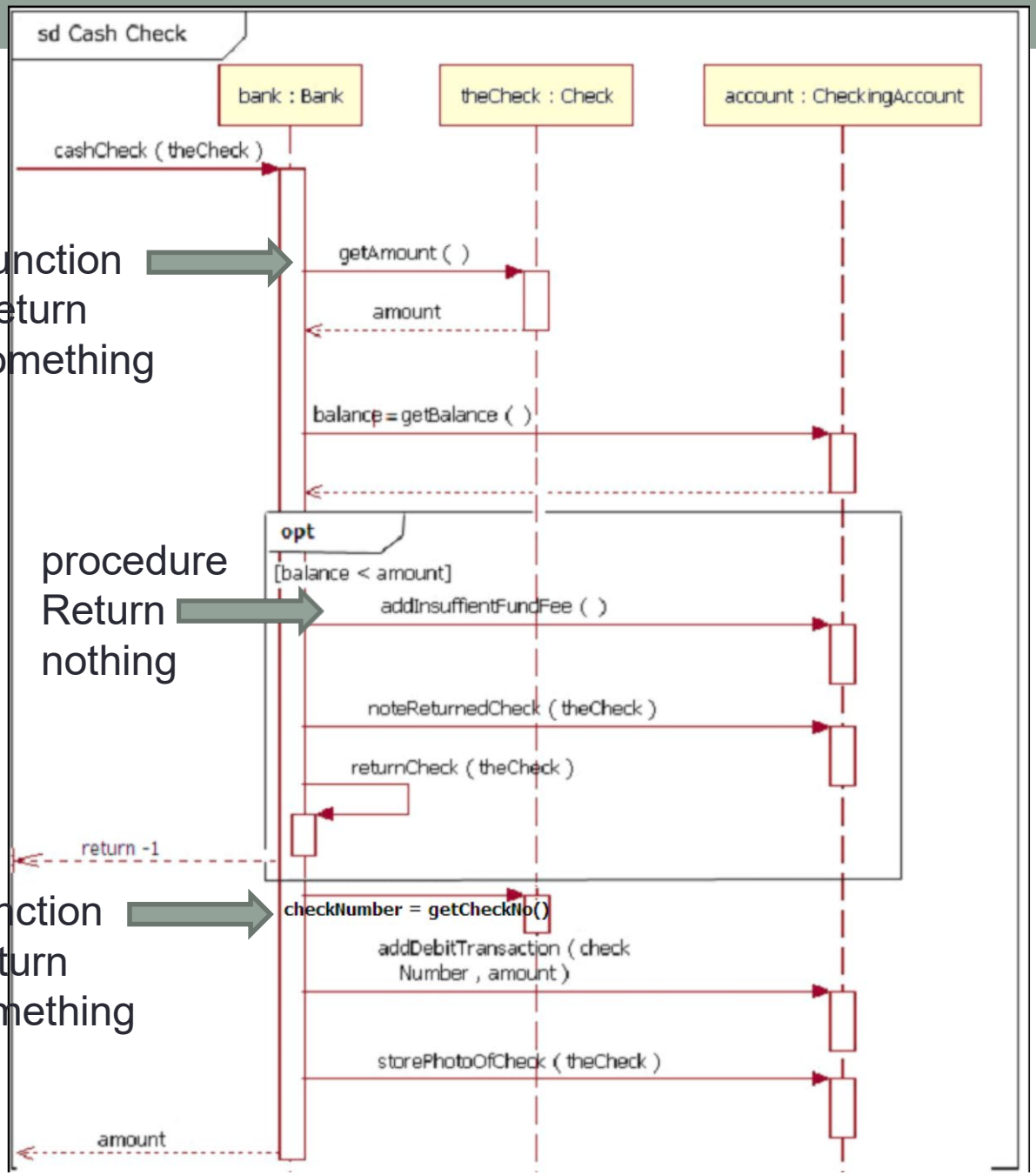
Make your codes COMPILABLE!



Sequence diagram

Before colon is the reference name and after the colon is the class name.

Q2



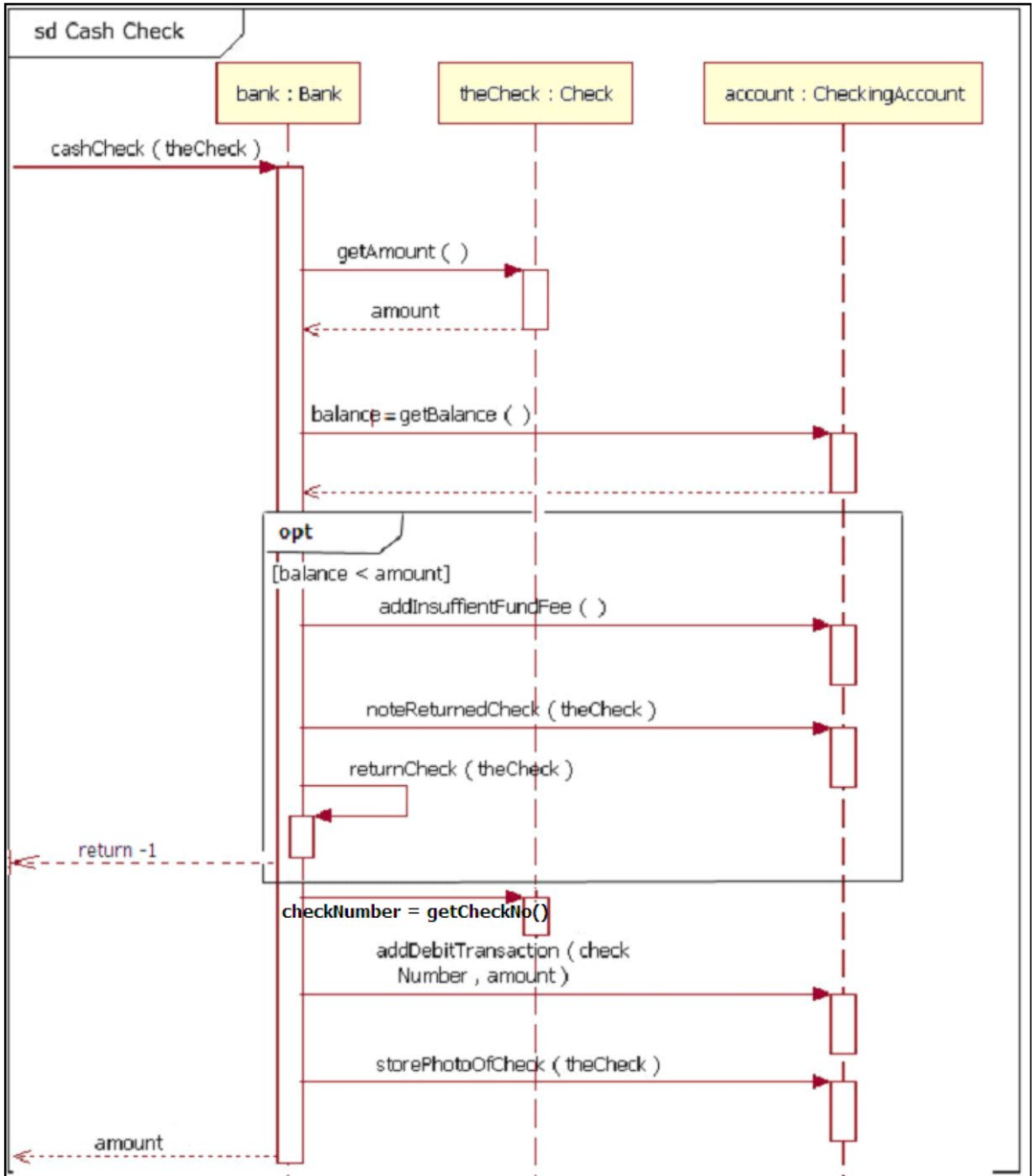
Function Return something

procedure Return nothing

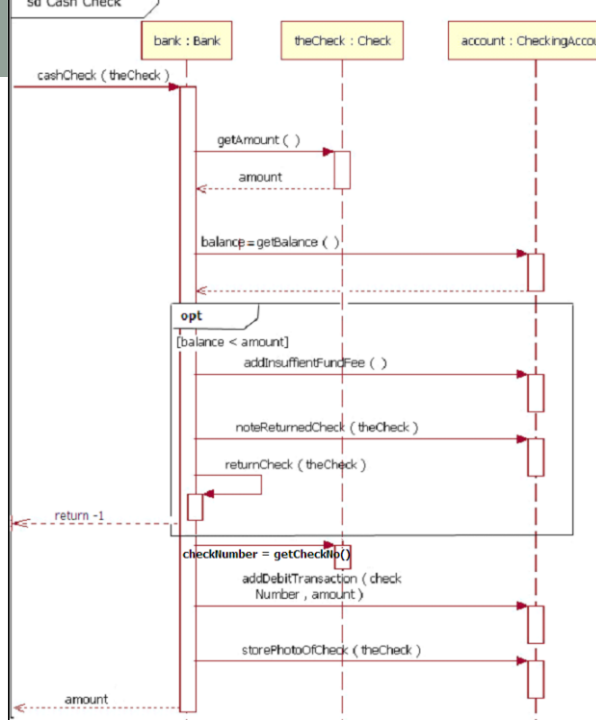
Function Return something

Self-call: call a method defined in the class itself

opt is used to describe optional step in workflow.(if statement)


sd Cash Check
bank : Bank
theCheck : Check
account : CheckingAccount
cashCheck (theCheck)
getAmount ( )
amount
balance = getBalance ( )
opt
[balance < amount]
addInsuffientFundFee ( )
noteReturnedCheck (theCheck)
returnCheck (theCheck)
return -1
checkNumber = getCheckNo()
addDebitTransaction (check Number , amount)
storePhotoOfCheck (theCheck)
amount

A2




```
public class Bank {
  private CheckingAccount account ;
  public double cashCheck(Check theCheck) {
        double amount = theCheck.getAmount();
        double balance = account.getBalance() ;
        if (balance < amount) {
              account.addInsuffientFundFee() ;
              account.noteReturnedCheck(theCheck);
              returnCheck(theCheck);
              return -1 ;
        }
        int checkNumber = theCheck.getCheckNumber() ;
        account.addDebitTransaction(checkNumber, amount);
        account.storePhotoOfCheck(theCheck);
        return amount;
}
  private void returnCheck(Check theCheck)  {  } }
```

A2

```
public class Check {
  public double getAmount() {
        return 100;  // any valid double
  }
  public int getCheckNumber() {
        return 1234; // any valid int
  }
}
```

A2

```
public class CheckingAccount {


  public double getBalance() {
        return 100;  // any valid double
  }
  public void addInsuffientFundFee()  {   } // void
  public void noteReturnedCheck(Check theCheck) {   }
  public void addDebitTransaction(int checkNumber, double
  amount) {   }
  public void storePhotoOfCheck(Check theCheck) {   }
}
```

Q2

- Draw the UML Sequence Diagram to show the flow of a user accessing its library items.

When a user wants to access a document or a folder, his/her account's capability is checked against an access level required by the document or folder.

If a user has an account, he/she can logon to the library. The user with the right capability can open, delete, and copy a folder or a document. A document can be also edited by the user.

https://ntu-sp.primo.exlibrisgroup.com/discovery/jsearch?query=any,contains,object&tab=jsearch_slot&vid=65NTU_INST:65NTU_INST&offset=0&journals=any,object

NANYANG TECHNOLOGICAL UNIVERSITY SINGAPORE

START A NEW SEARCH | FIND DATABASES | FIND JOURNALS | FIND EXAM PAPERS | REQUEST FOR DOCDEL/ILL | RECOMMEND BOOK PURCHASE | ...

Journal Search: object

Sign in to get complete results and to request items — Sign in | DISMISS

PAGE 1 *18 Results*

Refine my results

Sort by: Relevance

Limit to
- Physical Items in Library
- Online

Journals by category
- Arts, Architecture & Applied Arts
- Business & Economics
- Earth & Environmental Sciences
- Engineering & Applied Sciences
- General
- Health & Biological Sciences

1 JOURNAL
Object
View Online

2 JOURNAL
Technology of object-oriented languages and systems : TOOLS.
TOOLS (Conference); IEEE Computer Society.
1989
View Online

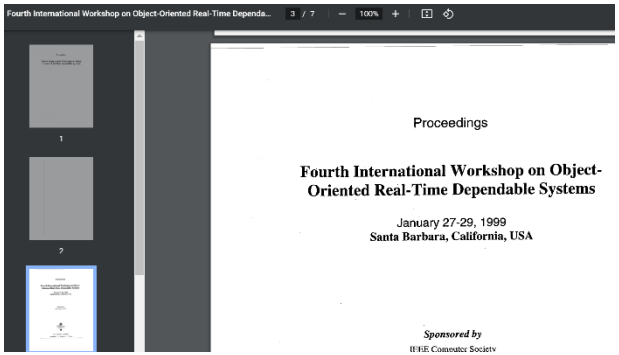
3 JOURNAL
OOPSLA: Conference on Object Oriented Programming Systems Languages and Applications
View Online

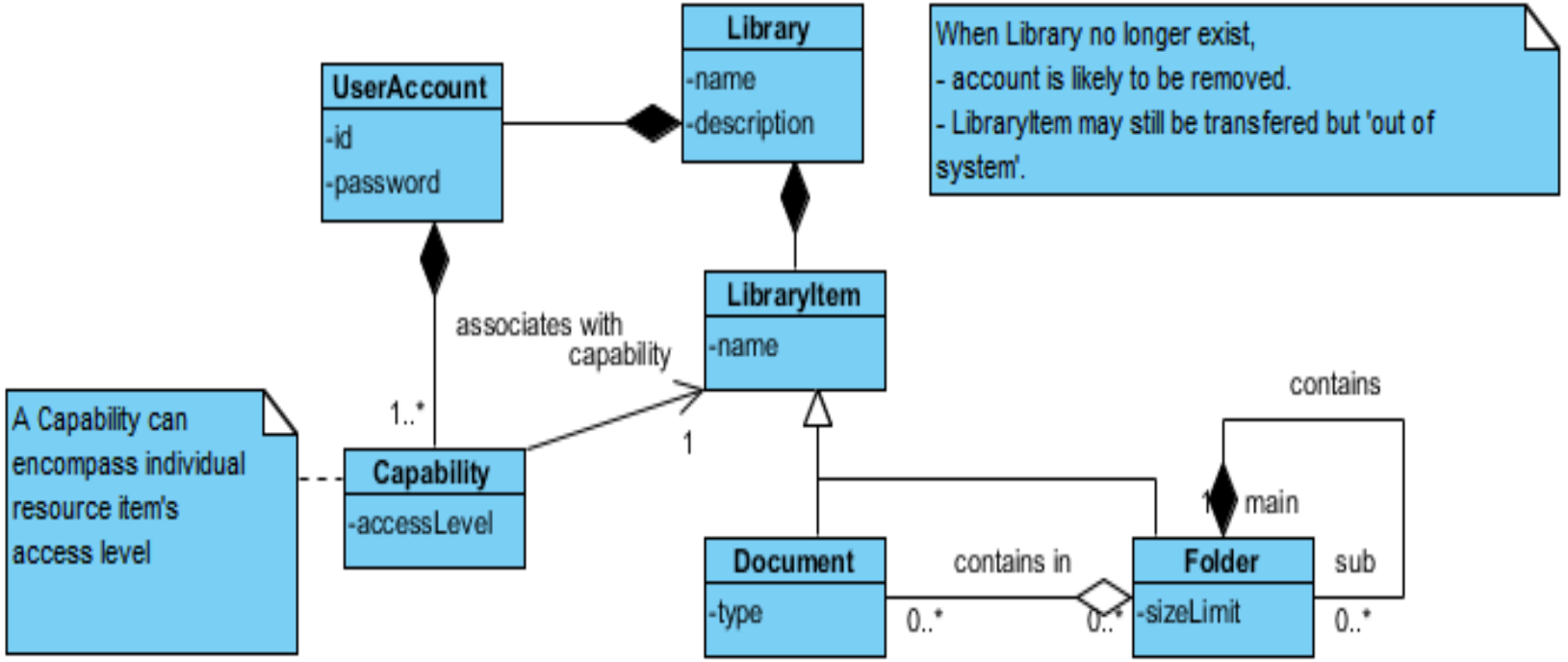
4 JOURNAL
International Workshop on Object-Orientation in Operating Systems : [proceedings].
International Workshop on Object Orientation in Operating Systems : IEEE Computer Society : IEEE

1999 Proceedings. Fourth International Workshop on Object-Oriented Real-Time Dependable Systems
Location: Santa Barbara, CA, USA

Fourth International Workshop on Object-Oriented Real-Time Dependable Systems (Cat. No.PR00101)
Publication Year: 1999 , Page: i

(215 Kb) ©





When a user wants to access a document or a folder, his/her account's capability is checked against an access level required by the document or folder.

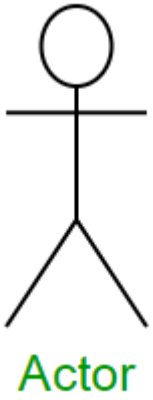
If a user has an account, he/she can logon to the library. The user with the right capability can open, delete, and copy a folder or a document. A document can be also edited by the user.

Step 1 - Define who will initiate the interaction

- Draw an actor on the diagram to specify who kick starts the interaction within a system.

Actors

An actor in a UML diagram represents a type of role where it interacts with the system and its objects. It is important to note here that an actor is always outside the scope of the system we aim to model using the UML diagram.



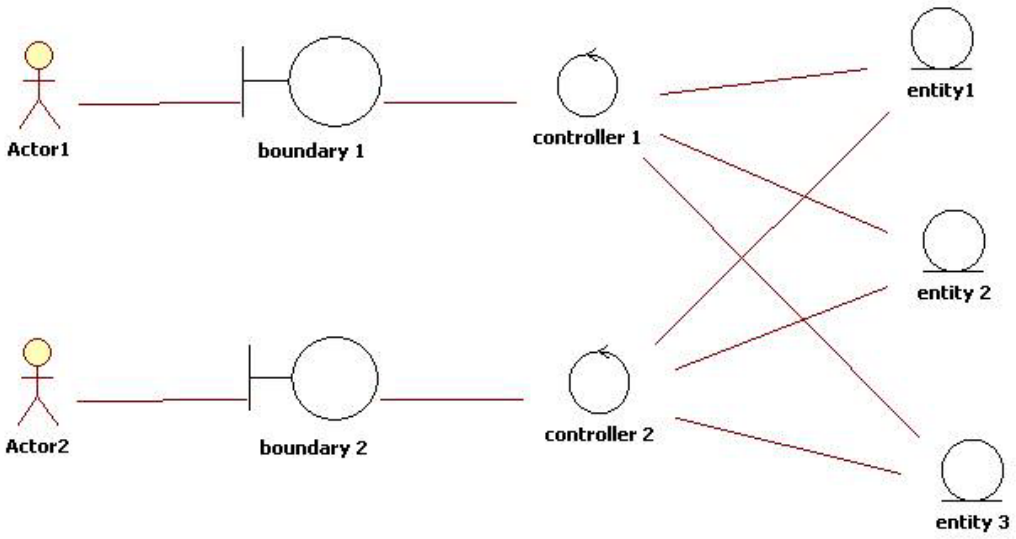
notation symbol for actor

When a user wants to access a document or a folder, his/her account's capability is checked against an access level required by the document or folder.

If a user has an account, he/she can logon to the library. The user with the right capability can open, delete, and copy a folder or a document. A document can be also edited by the user.

The analysis object model instantiates the Entity-Control-Boundary Pattern (ECB)

- ECB partitions the system into three types of classes: entities (data), controls, and boundaries (interface).



Entities (Database, Dataset)

Entities are classess representing system **data**: Customer, Product, Transaction, Cart, etc.

Boundaries

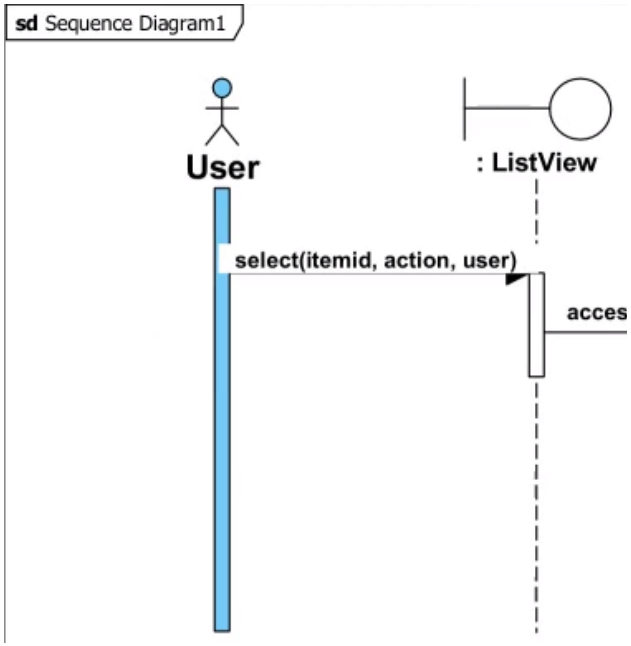
Boundaries are classess that **interface** with system actors: UserInterface, DataBaseGateway, ServerProxy, etc.

Controls

Controls are classes that mediate between boundaries and entities. They orchestrate the execution of commands coming from the boundary by interacting with entity and boundary objects. (**manager**)

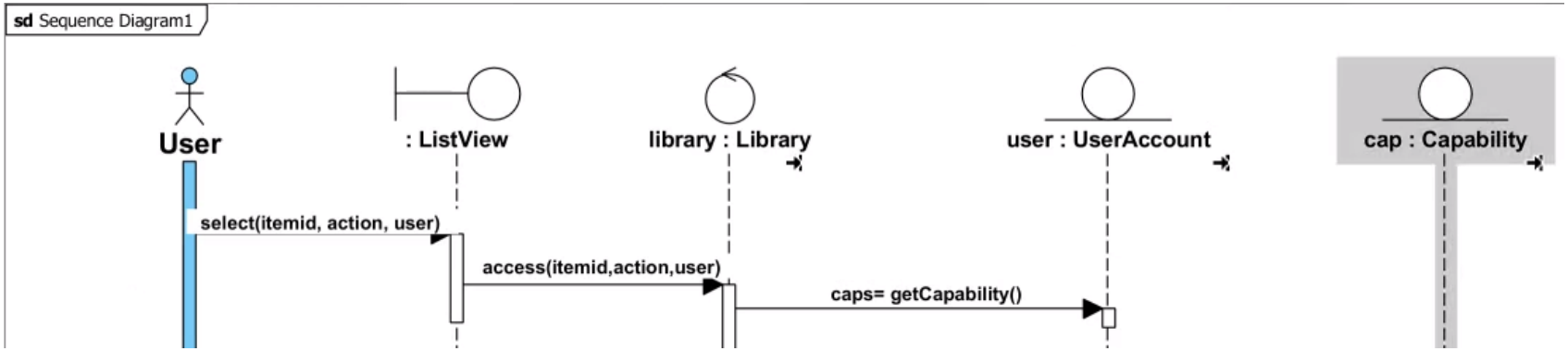
Step 2 - Draw the first message to a sub-system

- Specify the message sent from the actor who begins the interaction to the first point of contact in the system.



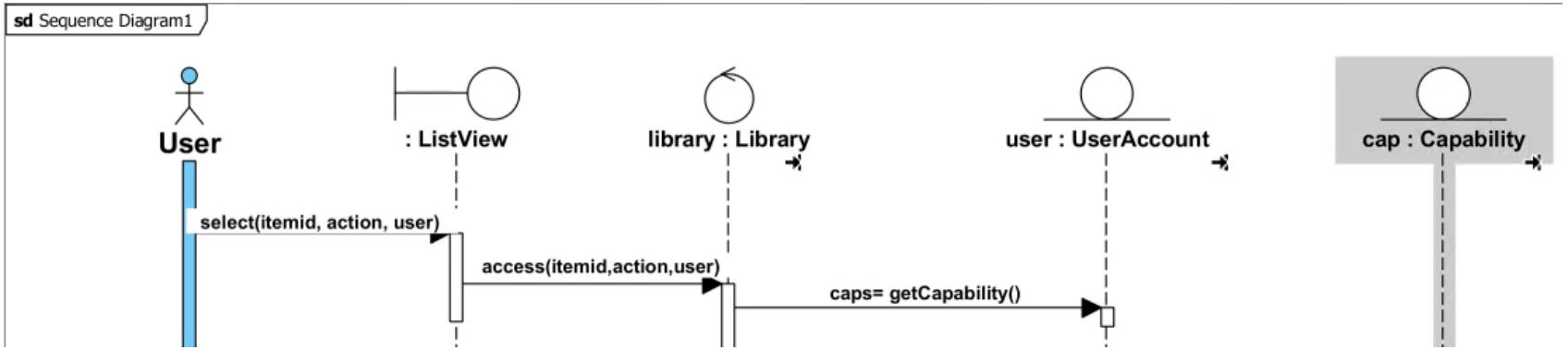
Step 3 - Draw message to other sub-systems

- Send other messages between objects (i.e. lifelines) in the system.



Step 4 - Draw reply message to actor

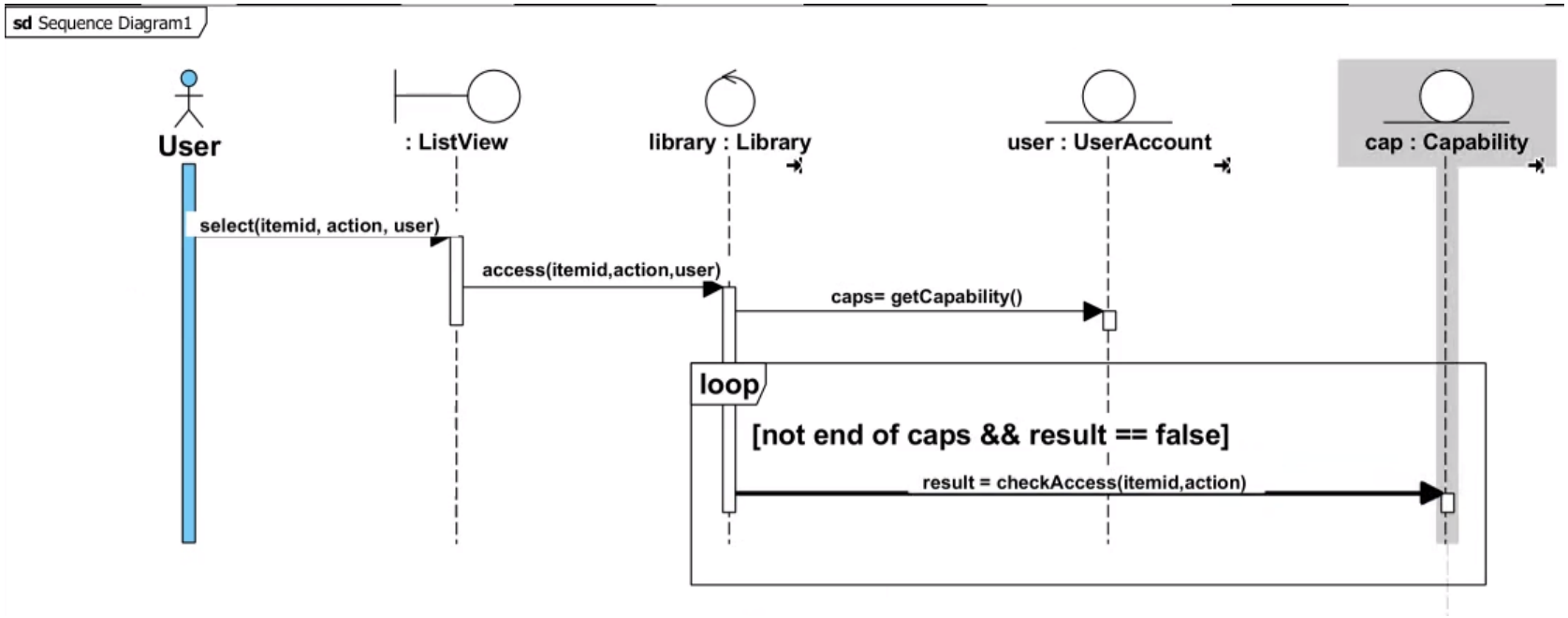
- Send reply messages back to the original callers upon receiving their messages.



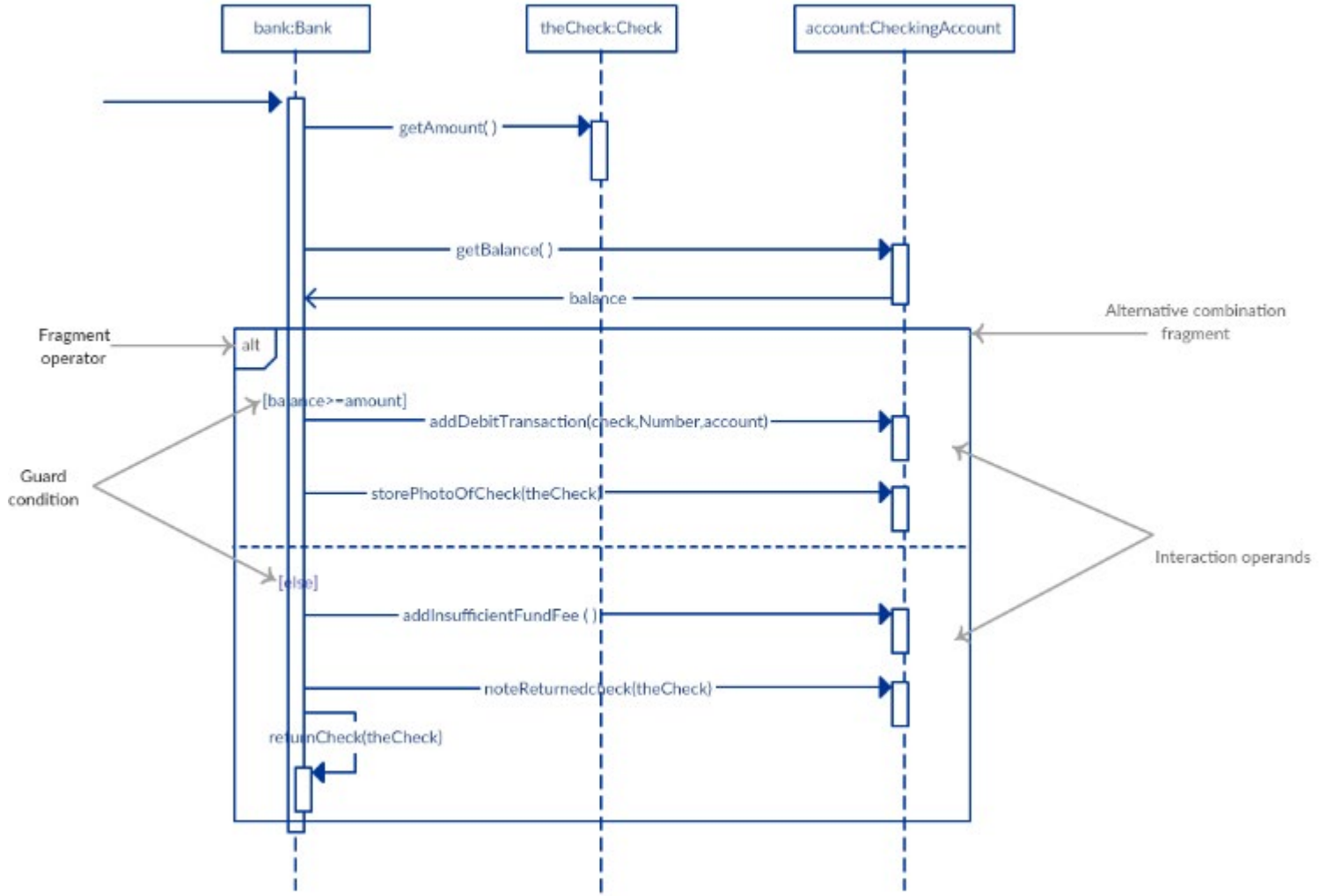
Loops

	Object		
Subject	File1	File2	Folder1
Alice	rw	r	rw
Bob	r	r	r
Cindy	rw	rw	r
Dan	..	..	..

Loop fragment is used to represent a repetitive sequence. Place the words ‘loop’ in the name box and the guard condition near the top left corner of the frame.



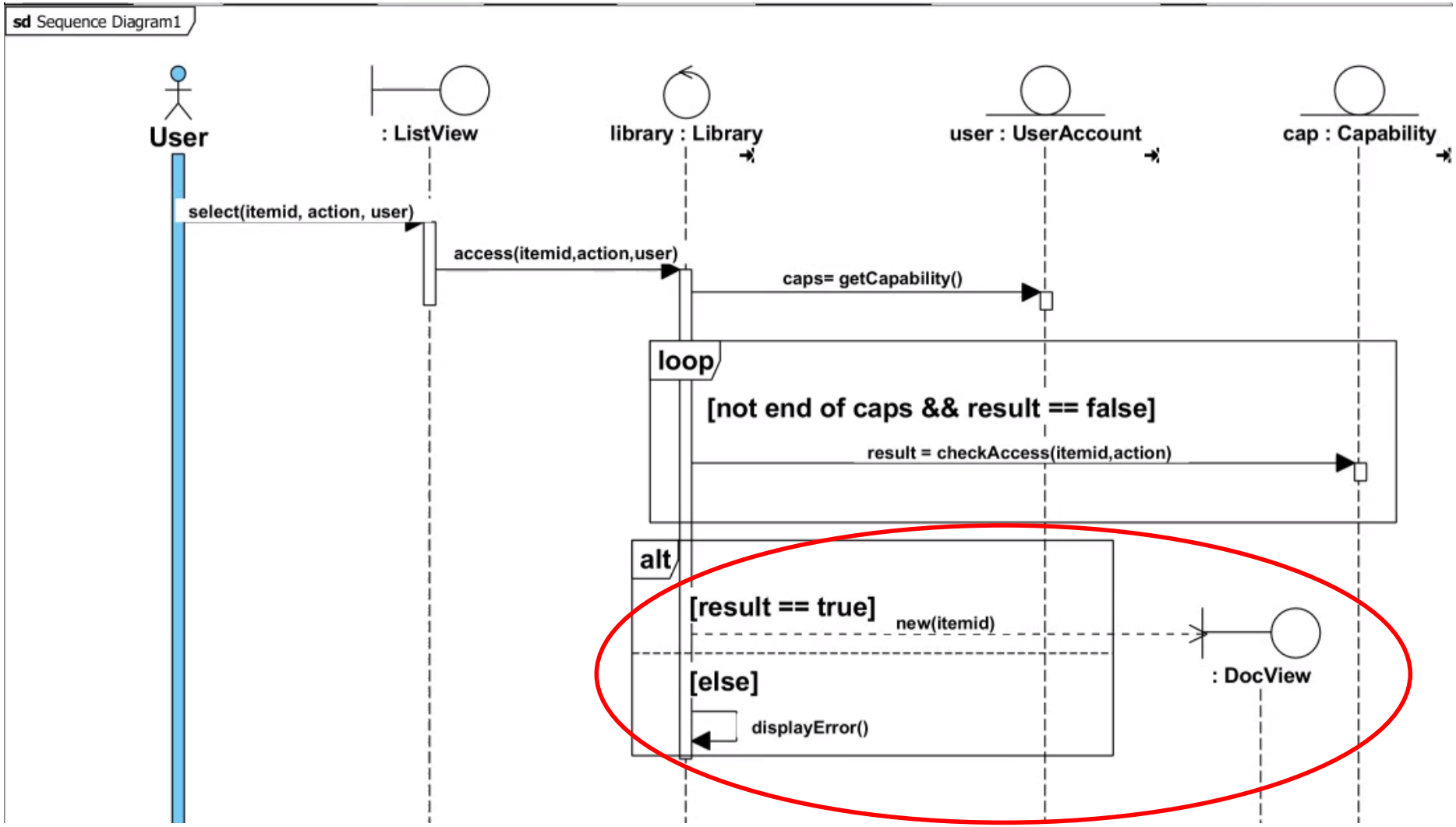
Alternatives cont'



Participant creation message

- Objects do not necessarily live for the entire duration of the sequence of events. Objects or participants can be created according to the message that is being sent.

- The dropped participant box notation can be used when you need to show that the particular participant did not exist until the create call was sent. If the created participant does something immediately after its creation, you should add an activation box right below the participant box.



Version 1: One control class

Version 2: Two control classes to share the work